

Projeto Prático: Sistema Distribuído de Telemetria e Controle de Tráfego Urbano Crítico (SD-Trafego)

Disciplina: Sistemas Distribuídos

Formato de Entrega: Seminário Prático com Demonstração de Falhas Ao Vivo (10 minutos)

Trabalho em Grupo: 1 entrega por grupo - nota 30% entrega Grupo e 70% Individual (apresentação)

Prazo de Entrega: Até 16/07/2026

1. Objetivo Geral

Desenvolver um protótipo de sistema distribuído descentralizado para monitoramento e controle de tráfego urbano em tempo real utilizando o modelo arquitetural baseado em eventos (Publish-Subscribe). O sistema deve ser totalmente containerizado, resiliente a falhas parciais, partições de rede, assincronismo extremo e deriva de relógios, aplicando rigorosamente os conceitos de processos, comunicação, coordenação, consistência e tolerância a falhas.

2. Requisitos Técnicos Obrigatórios e Tecnologias

- Linguagem de Programação: Python 3.x (uso restrito de bibliotecas nativas ou middlewares homologados; proibido o uso de frameworks que resolvam os problemas distribuídos de forma opaca).
- Infraestrutura: Ambiente multicontêiner isolado gerenciado via Docker Compose.
- Middleware Pub-Sub: Um broker centralizado (Apache Kafka ou RabbitMQ) rodando em contêiner próprio.

3. Arquitetura do Sistema e Divisão de Papéis

O sistema será composto por 4 módulos principais. Cada estudante do grupo será o engenheiro responsável por um módulo, devendo defendê-lo tecnicamente no seminário:

- Módulo 1: Infraestrutura de Rede e Injeção de Caos Responsável pela orquestração do docker-compose, topologia de rede interna e garantia de isolamento. Deve implementar scripts de Chaos Engineering usando ferramentas de controle de tráfego de rede (como tc/netem no Linux ou contêineres proxy) para injetar ativamente perda de pacotes e latência variável na comunicação com o Broker.

- **Módulo 2: Produtores (Publishers) e Tempo Lógico**
Responsável pelo desenvolvimento dos nós sensores de tráfego em Python. Deve simular múltiplas instâncias que enviam dados de fluxo de veículos. Como a rede será instável, deve implementar Relógios Lógicos de Lamport ou Relógios Vetoriais anexados a cada mensagem enviada, garantindo a ordenação causal dos eventos no destino.
- **Módulo 3: Consumidores (Subscribers) e Consenso de Liderança**
Responsável pelas instâncias de atuadores (Semáforos Inteligentes). O controle dos semáforos exige um coordenador único. Deve implementar um algoritmo de Eleição de Líder customizado (baseado no algoritmo do Bully ou Ring) diretamente em Python via mensagens de controle.
- **Módulo 4: Estado, Tolerância a Falhas e Recuperação**
Responsável pela persistência e consistência. Deve implementar mecanismos de detecção de falhas de nós (*Heartbeats*) e um sistema de Checkpoints locais. Ao sofrer uma queda abrupta (docker kill), o nó, ao ser reiniciado pelo Docker, deve recuperar seu estado anterior de forma transparente sem corromper o histórico global.

4. Restrições Avançadas e Cenários de Validação

Para obter aprovação de 100%, o sistema obrigatoriamente deve mitigar e resolver os três cenários de caos abaixo, desenvolvidos para testar a consistência do ecossistema distribuído:

Restrição A: Rede Não-Confíável e Inversão Temporal

- O Cenário: A rede injetará latências flutuantes de 10ms a 4000ms e 5% de perda de pacotes.
- O Desafio: Mensagens enviadas pelo Sensor A às 14:00:01 podem chegar após mensagens enviadas pelo Sensor B às 14:00:03.
- Validação: Os Subscribers não podem processar os dados na ordem em que chegam na rede (ordem de chegada física), mas sim garantir a execução estrita baseada na ordenação causal computada pelos relógios lógicos.

Restrição B: Partição de Rede e Prevenção de Split-Brain

- O Cenário: Durante a execução, o script de caos dividirá a rede Docker em duas sub-redes isoladas. Duas instâncias de semáforo ficarão de um lado e as outras duas do outro.
- O Desafio: Algoritmos tradicionais de eleição criariam dois líderes simultâneos (um em cada partição), o que causaria acidentes de trânsito simulados catastróficos.
- Validação: O algoritmo de eleição deve obrigatoriamente exigir Quórum Absoluto (Maioria Estrita). A partição que não obtiver mais de 50% dos nós totais ativos deve

se autoexcluir e entrar em modo de segurança, impedindo a existência de múltiplos líderes (*split-brain*).

Restrição C: Deriva de Relógio Físico Isolado

- O Cenário: Cada contêiner de sensor introduzirá uma taxa de erro artificial em seu relógio de sistema (ex: acelerar ou atrasar artificialmente em relação ao tempo real do host). É proibido consultar servidores NTP externos da internet.
- O Desafio: Os dados precisam de correlação de tempo físico para relatórios de tráfego macrourbano.
- Validação: O grupo deve codificar manualmente uma variação do Algoritmo de Cristian ou Algoritmo de Berkeley utilizando o próprio middleware Pub-Sub para calcular o *offset* de tempo de cada contêiner e sincronizar os relógios internamente de forma distribuída.

5. Critérios de Avaliação do Seminário (Apresentação Prática)

A nota será composta pelo domínio teórico dos conceitos aplicados e pela execução bem-sucedida da demonstração ao vivo:

1. Mapeamento Teórico (25%): Demonstração clara nos slides de onde estão localizados os conceitos dos Capítulos 2, 3, 4, 5, 6 e 7 do livro do Van Steen no código fonte do projeto.
2. Prova de Ordenação Causal (25%): Exibição dos logs provando que mensagens que sofreram atrasos extremos de rede foram reordenadas e processadas na sequência lógica correta.
3. Prova de Resiliência e Quórum (25%): Executar um comando docker kill no líder atual ou isolará uma sub-rede. O sistema deve detectar a falha e eleger um novo líder (se houver quórum) ou travar com segurança (se não houver), registrando tudo em logs claros.
4. Qualidade da Arquitetura (25%): Legibilidade do código Python, tratamento de exceções de rede (TimeoutError, ConnectionRefusedError) e correto isolamento dos papéis de cada integrante do grupo.

Desejo a todos um bom projeto. A engenharia de sistemas distribuídos se prova no comportamento do sistema sob falhas, e não no cenário ideal!